파이썬 핵심 라이브러리와 Numpy

심선영 교수

강의 목표

❖ 파이썬 라이브러리를 이해한다.

❖ 데이터 분석에서 사용되는 핵심 라이브러리의 종류를 익힌다.

❖ Numpy 라이브러리를 실습을 통해 익힌다.

강의 순서

- ❖ 파이썬 라이브러리
- ❖ Numpy

파이썬 라이브러리

프로그램의 구조

모듈, 라이브러리

- 프로그램에서 코드를 체계적으로 모아서 관리

명령어

```
a = 32
b = 10
a*b
```

```
320
```

- 기본 명령어

함수

```
def mul (a, b):
  return a*b
mul (32, 10)
```

```
320
```

- 반복 되는 명령어 묶음

모듈

```
# arithmatic module
def add (a,b) :
  return a+b
def substract(a, b):
  return a-b
```

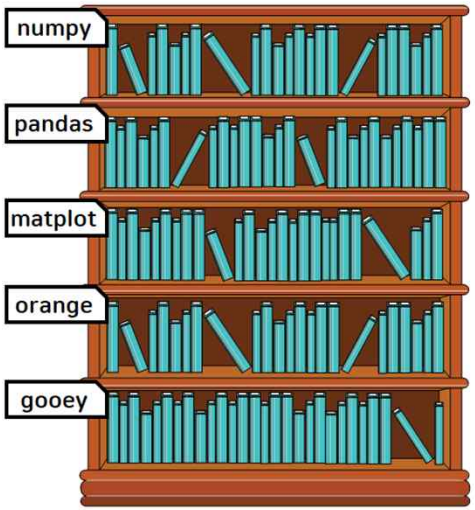
- 유사한 함수와 클래스를 묶어 **모듈**을 구성

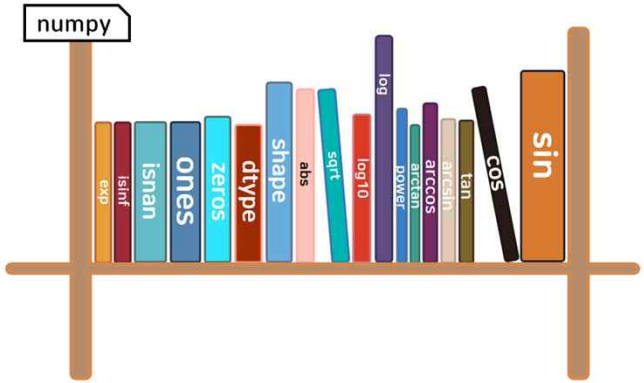
라이브러리 (패키지)

```
* 머신러닝 라이브러리
* 그래프 라이브러리
* 통계 라이브러리
...
```

- 기능별 로 모듈을 묶어 **패키지** 구성.
- 공유하기 용이하게 정리된 형태가 **라이브러리**

라이브러리(Library)





라이브러리(Library)

데이터 과학 관련 파이썬 대표 라이브러리

NumPy 넘파이 : 데이터 연산

- **배열** 자료구조 생성과 빠른 연산

```
[[1, 2],      [[100, 200],      [[101, 202],
 [3, 4],  +    [300, 400],  =    [303, 404],
 [5, 6]]       [500, 600]]       [505, 606]]
```

pandas 판다스 : 데이터 관리

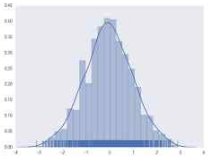
- 표 형태의 **데이터프레임** (엑셀과 유사)

	Name	Age	Department
0	Jim	26	Sales
1	Dwight	28	Sales
2	Angela	27	Accounting
3	Tobi	32	Human Resources

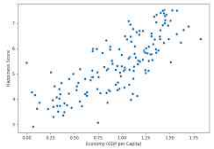
matplotlib 맷플랏립 , seaborn 씨본 : 그래프 (시각화)

- 데이터 이해를 위한 **그래프** 생성

히스토그램

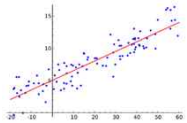


산포도



scikit-learn 싸이킷런 : 머신러닝

- 회귀 분석 등 **머신러닝** 수행



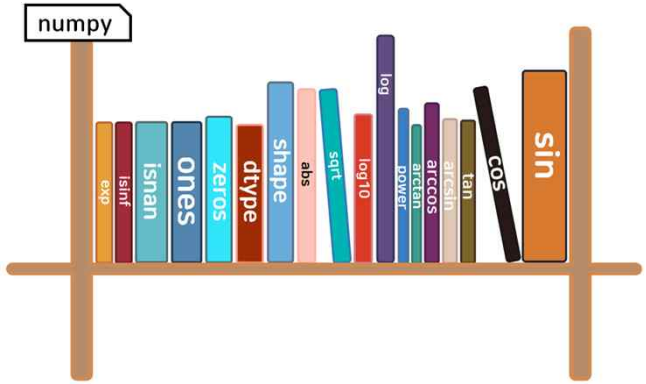
$$y = ax + b$$

<출처> https://numpy.org/, https://pandas.pydata.org/, https://matplotlib.org/, https://scikit-learn.org/stable/

라이브러리(Library)

❖ 라이브러리 import

- **import** 라이브러리 이름 **as** 별명
- **from** 라이브러리 이름 **import** 함수/클래스 이름
- **'.'** 소속을 의미



❖ 라이브러리 사용법

```
import numpy as np
np.cos(3.141592)
```

```
-0.9999999999997864
```

Numpy

Numpy란?

❖ 과학, 공학 분야의 계산에서 가장 많이 활용되는 기본 라이브러리

- https://numpy.org/

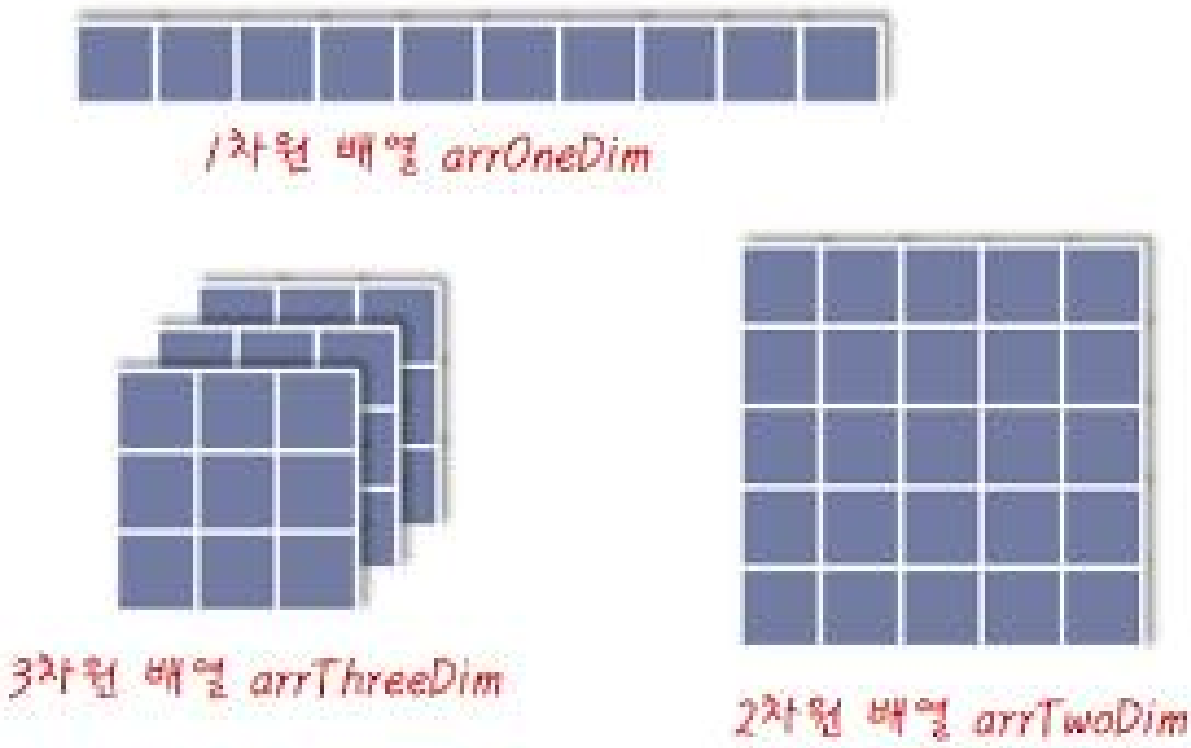
❖ 특징

- 다차원 배열 처리가 가능
- 기능적으로 뛰어나며, 수행 속도가 빠르다
- 수치 연산 라이브러리의 기반

Numpy 배열

❖ 배열 (array)란?

- 1차원, 2차원, 고차원 상에 연속적으로 데이터를 둔 것



Numpy 배열

❖ Numpy배열 (ndarray)란?

- 파이썬 기본 라이브러리에서 리스트를 통해 배열을 사용 가능하지만, 리스트는 복잡한 배열연산에 적합하지 않음
- 파이썬으로 구현된 머신러닝 딥러닝 코드는 일반적으로 numpy 배열을 사용
- Numpy의 array함수를 통해 리스트를 배열로 변환 가능
- **같은 종류의 데이터**만 넣을 수 있음.
  - (차이) 리스트 : 다양한 종류의 데이터 입력 가능

3	4	1	2	8	2	4	9

[그림] 1차원 배열

```
import numpy as np
x = np.array([1, 2, 3, 4])
```

3	4	1	2
8	2	4	9

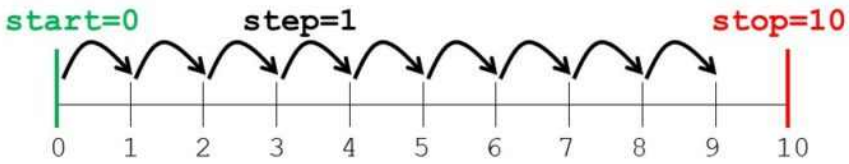
[그림] 2차원 배열

```
x = np.array([2,3,4],[1,2,5] ] )
```

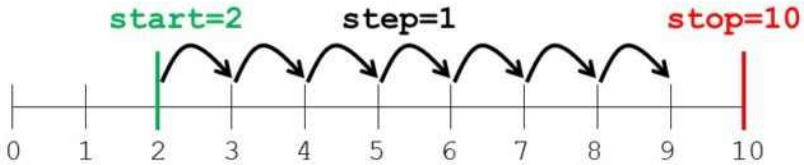
함수 사용하기 - arange

❖ np.arange 주의!

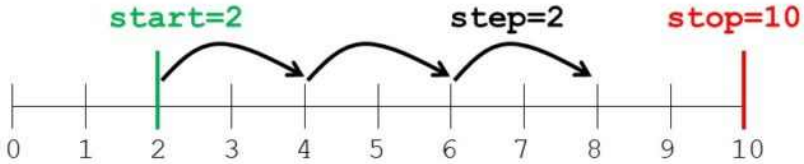
```
>>> np.arange(10)
[0 1 2 3 4 5 6 7 8 9]
```




```
>>> np.arange(2, 10)
[2 3 4 5 6 7 8 9]
```




```
>>> np.arange(2, 10, 2)
[2 4 6 8]
```



2차원 배열

❖ x 는 2x5 배열

- x = arrange[10]
- x = x.reshape(2,5)

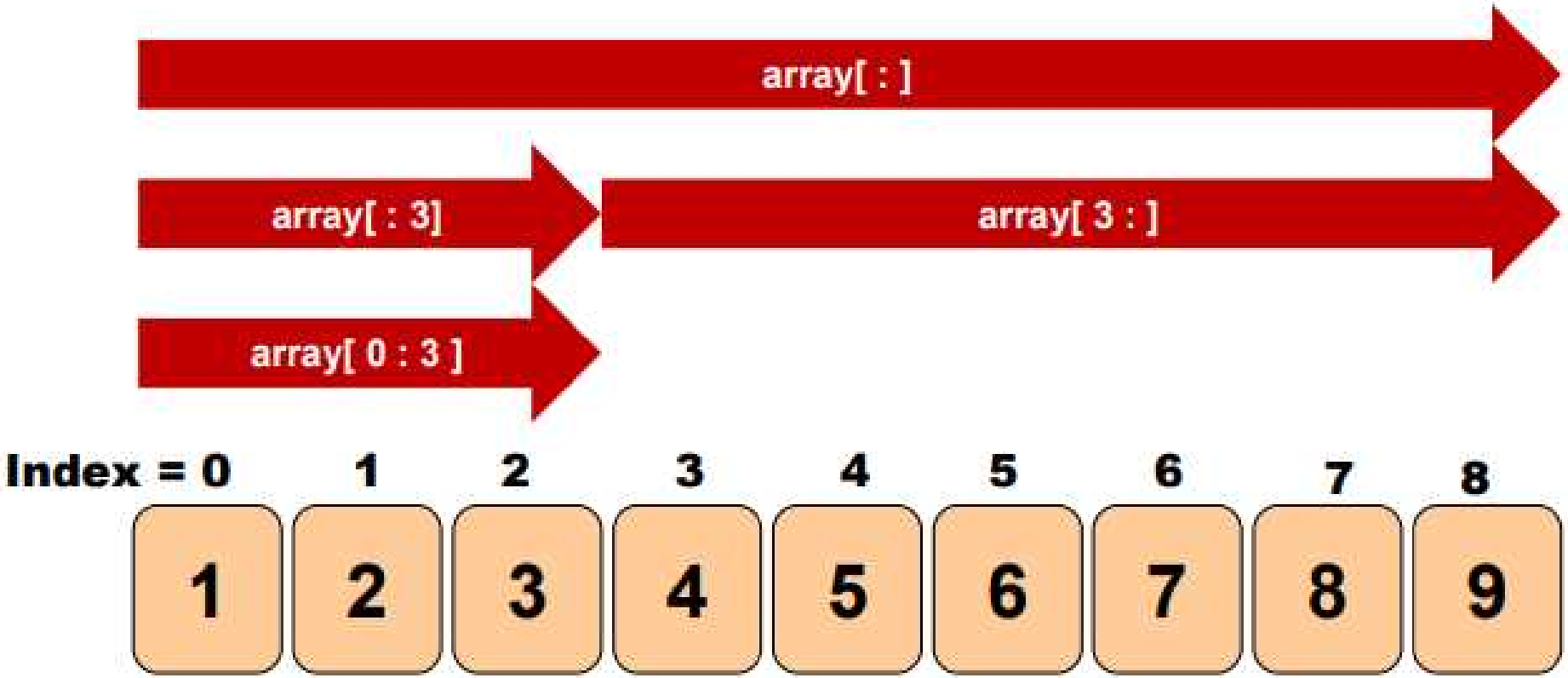
	[0]	[1]	[2]	[3]	[4]
[0]	0	1	2	3	4
[1]	5	6	7	8	9

열 인덱스

행 인덱스

a[1,3]

슬라이싱



2차원 슬라이싱

array2d[0:2, 0:2]

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

array2d[1:3, 0:3]

1	2	3
4	5	6
7	8	9

array2d[1:3, :]

1	2	3
4	5	6
7	8	9

2차원 슬라이싱

array2d[:, :]

1	2	3
4	5	6
7	8	9

array2d[:2, 1:]

1	2	3
4	5	6
7	8	9

array2d[:2, 0]

1	2	3
4	5	6
7	8	9

Numpy Boolean Indexing

```
import numpy as np
```

```
x = np.array([1,2,3,4,5,6,7,8,9,10])
```

```
x%2 == 0
```

```
array([False,  True, False,  True, False,  True, False,  True, False,
       True])
```

- 짝수만 나오도록 boolean indexing

```
x[x%2 == 0]
```

```
array([ 2,  4,  6,  8, 10])
```

- 짝수의 갯수

```
len(x[x%2 == 0])
```

```
5
```

- 짝수값들의 평균

```
np.mean(x[x%2 == 0])      #그냥 mean이 아니고 np.mean()!
```

```
6.0
```

Numpy 주요 함수

❖ [생성]

- np.arange(12).reshpe(3,4) = np.arrange(12).reshpe(-1,4) = np.arrange(12).reshpe(3,-1)
- np.zeros((4, 5)), np.ones((4, 5))
- np.empty((4, 5)), np.full((4, 5), 7)
- np.eye(5)

❖ [random sampling]

- np.random.rand(4,5) : [0,1)에서 (4,5)형태로 랜덤 샘플링 (실수)
- np.random.randn(4,5) : 표준정규분포에서 (4,5)형태로 랜덤 샘플링 (실수)
- np.random.normal(10, 100, size=(4, 5)): 평균10, 분산 100인 정규분포에서 (4,5)형태로 랜덤 샘플링 (실수)
- np.random.randint(100, 4,5) : [0, 100) 범위에서 (4,5)형태로 랜덤 샘플링 (정수)
- np.random.choice(100, size=(4, 5)) : [0, 100) 범위에서 (4,5)형태로 랜덤 샘플링, 복원 추출이 default (정수)

Numpy 주요 함수

❖ [연산]

- np.add (+)
- np.substract (-)
- np.multiply (*)
- np.divide (/)
- np.matmul (@)

❖ [통계]

- np.mean() : 평균
- np.var(), np.std() : 분산, 표준편차
- np.median(): 중간값
- np.min(), np.max(): 최대값, 최소값
- np.argmax() : 최대값의 index(1D-array로 간주하여)

Wrap-Up